4. Preguntas de análisis

Al finalizar la implementación, respondan las siguientes preguntas:

1. ¿Qué es la exclusión mutua en el contexto de este problema?

   Es la propiedad fundamental de la programación concurrente y distribuida que garantiza que un recurso compartido crítico (en este caso, un palillo) solo pueda ser accedido y utilizado por un único proceso (filósofo) a la vez.

2. ¿Cuáles son los recursos compartidos? Son los palillos

3. ¿Por qué dos filósofos contiguos no pueden comer simultáneamente?

   Porque hay 5 palillos y 5 filósofos. Cada filósofo tiene a su disposición 2 palillos, pero estos los comparte con los filósofos contiguos. Si un filósofo come, agarra sus 2 palillos cercanos y con esto le quita el palillo derecho a su compañero de la izquierda y el izquierdo al de la derecha; con esto, ambos filósofos se quedan sin un palillo y así no pueden comer.

4. ¿Puede haber dos filósofos comiendo al mismo tiempo? Justifique su respuesta.

   Sí, pero no deben ser contiguos. Hay filósofos que no comparten palillos; al no ser vecinos significa que hay filósofos entre ellos. Como son 5, siempre tendrán una distancia de 2 filósofos de un lado y uno del otro lado. Por lo tanto, a un filósofo lo dejarían sin palillos (ya que cada filósofo tomaría un palillo de ese filósofo) y a los otros 2 con un solo palillo (ya que agarrarían los palillos de las orillas de cada nodo). Por lo tanto, cada uno de los dos filósofos que comen se quedaría con sus 2 palillos y podría comer.

5. ¿Cómo garantiza su implementación que un palillo no sea utilizado por dos filósofos simultáneamente?

   Cada palillo es un proceso independiente que gestiona su propio estado (`:libre` o `:ocupado`). Cuando recibe un mensaje {`:solicitar, filosofo`} en estado `:libre`, responde con `:recibido` y cambia inmediatamente su estado a `:ocupado`. Si llega otra solicitud mientras está ocupado, esta se queda en el buzón del proceso del palillo y no se procesa hasta que el palillo reciba el mensaje `:liberar` y vuelva a estar libre.

6. ¿Qué función cumplen los procesos que representan a los palillos?

   Su función es mantener el control del estado del palillo, procesan las solicitudes de los filósofos una a una a través de su buzón de mensajes y garantizan el acceso exclusivo para evitar condiciones de carrera.

7. ¿Qué función cumplen los mensajes utilizados por los filósofos? Al no existir memoria compartida son el único mecanismo de comunicación y coordinación entre los procesos en Elixir. Solicitan un recurso ({`:solicitar, self()`}), y reciben la confirmación de asignación (`:recibido`) y notifican la liberación del palillo ({`:liberar, self()`}).

8. ¿Qué ocurriría si un filósofo obtiene solamente uno de los dos palillos que necesita?

   Se queda bloqueado en la instrucción `receive` esperando la confirmación del segundo palillo.Cuando los 5 filósofos tomaran un palillo al mismo tiempo y se quedaran esperando el segundo, el sistema caería en un interbloqueo, donde ningún proceso podría avanzar y nadie podria comer, por lo que todos morirían de hambre.

9. ¿Qué modificaciones serían necesarias para evitar que un filósofo pueda quedarse esperando indefinidamente por un palillo?

   - **Asimetría:** Los filósofos pares solicitan primero su palillo derecho y los impares el izquierdo, evitando la espera circular.
   - **Tiempo de espera:** Se agrega un bloque `after` en el `receive`. Si un filósofo no consigue el segundo palillo tras cierto tiempo, debe liberar el primero, esperar un tiempo aleatorio y volver a intentar.
   - **Proceso coordinador (Árbitro):** Implementa un proceso central que solo otorgue permiso de comer si ambos palillos están disponibles simultáneamente.